AWS / EWB CLOUD LABS / PROJECT 01

VPC Peering

Connect two VPCs. Prove private connectivity.

2 VPCs / 3 EC2 instances / 1 peering connection / 1 bastion host



STUDENT PROJECT REPORT

Name	Aman Kumar
Batch	_____
Project title	AWS VPC Peering: connect two VPCs and prove private connectivity
Region	Asia Pacific (Mumbai) / ap-south-1
Completion date	22 October 2026

Course: AWS Architect Associate (EWB Courses). Report file: AmanKumar_EWB_VPC_Peering.pdf

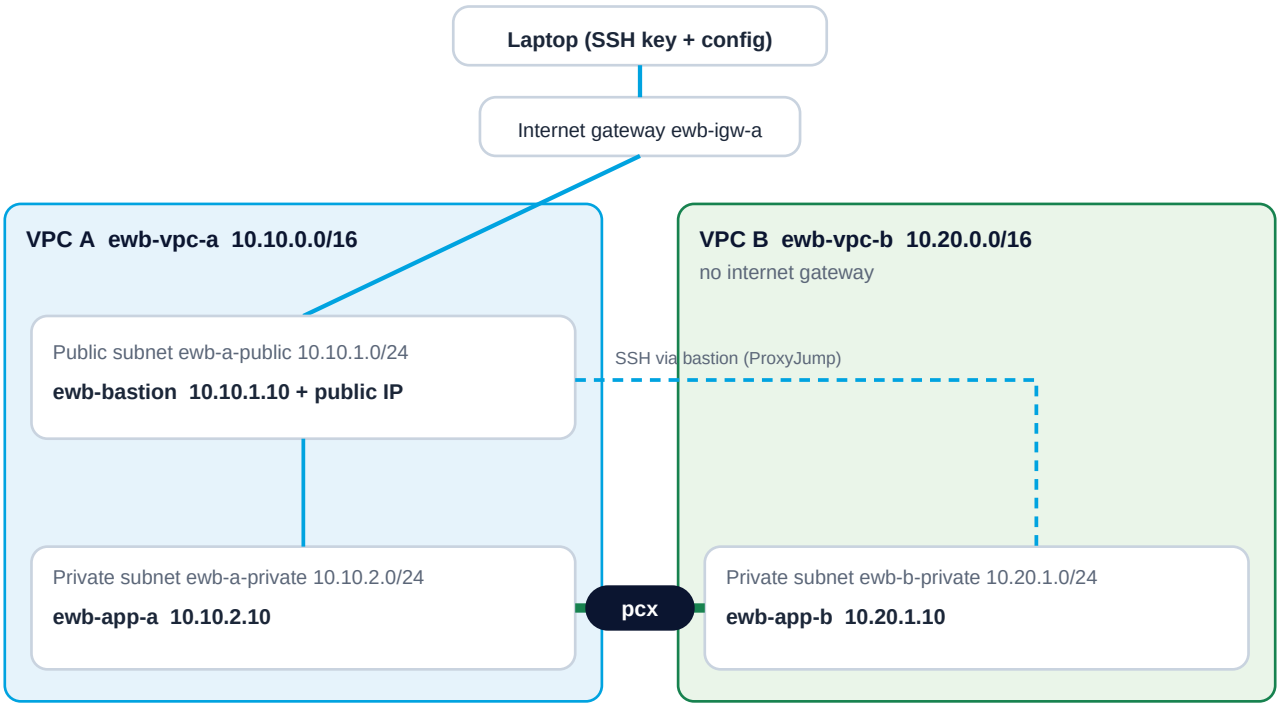
What this project builds

Two VPCs were created and connected with one VPC peering connection. Both application servers (App-A and App-B) stay private, with no public IPv4 address. A single bastion host with a public IP in VPC A is the only entry point, and administration of the private servers uses SSH ProxyJump through it. The project proves that App-A and App-B can exchange traffic over their private IPs through the peering connection, and shows a controlled failure and recovery.

Report contents (submission checklist)

Item	Section
Cover: name, batch, project title, Region, completion date	Cover
Architecture diagram and inventory with actual resource IDs and private IPs	2
Subnet CIDRs and all three route tables with subnet associations	3
Active peering connection and inbound rules of each security group	4
Baseline failure, forward and reverse ping, App-B SSH proof	5
Controlled ICMP failure and recovery with explanation	6
Forward and return path, cleanup evidence, what I learned	7

Network diagram and resource inventory



Administration path: laptop, then bastion, then App-A or App-B (SSH ProxyJump). Project traffic path: App-A and App-B over private IPs and the peering connection. Peering does not give VPC B internet access.

Resource	Actual ID	CIDR / address / note
ewb-vpc-a	`vpc-0c5f736c9fc30ad6f`	10.10.0.0/16
ewb-a-public (subnet)	`subnet-0fdc794f65aa19cba`	10.10.1.0/24
ewb-a-private (subnet)	`subnet-02bf637ddb0e27d36`	10.10.2.0/24, ap-south-1a
ewb-vpc-b	`vpc-0179eca8d4644d178`	10.20.0.0/16
ewb-b-private (subnet)	`subnet-0b32589cab30db11b`	10.20.1.0/24, ap-south-1c
ewb-igw-a	`igw-078d383a8a0d774d0`	Attached to ewb-vpc-a
ewb-rt-a-public	`rtb-086386f931fae0357`	Associated with ewb-a-public
ewb-rt-a-private	`rtb-044aead01aaaa28ff`	Associated with ewb-a-private
ewb-rt-b-private	`rtb-01e22e430e3341e37`	Associated with ewb-b-private
ewb-bastion	`i-0900dc99b7ca8db08`	10.10.1.10 + public IPv4 52.66.204.141, ap-south-1a
ewb-app-a	`i-00a4b627084dbaa6e`	10.10.2.10, no public IPv4, ap-south-1a
ewb-app-b	`i-022316462bf65ab90`	10.20.1.10, no public IPv4, ap-south-1c
ewb-a-b-peer	`pcx-016ad3738d94d964e`	Active, requester VPC A, accepter VPC B
ewb-sg-bastion	`sg-091643d4ea17fb83a`	VPC A
ewb-sg-app-a	`sg-0be57108d70d95160`	VPC A
ewb-sg-app-b	`sg-094eae3a31436b6a9`	VPC B
Key pair ewb-lab	`RSA, .pem`	Private key stays on my laptop and is not part of this report

Subnet CIDRs and the three route tables

VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16) do not overlap, which is required for peering. Each /24 subnet sits inside its own VPC range.

Subnet	VPC	CIDR	Type
ewb-a-public	ewb-vpc-a	10.10.1.0/24	Public (route to IGW)
ewb-a-private	ewb-vpc-a	10.10.2.0/24	Private (no internet route)
ewb-b-private	ewb-vpc-b	10.20.1.0/24	Private (no IGW in VPC B)

Route table	Destination	Target	Purpose
ewb-rt-a-public	10.10.0.0/16	local	Bastion to App-A
	0.0.0.0/0	igw-078d383a8a0d774d0	Bastion internet access
	10.20.0.0/16	pcx-016ad3738d94d964e	Bastion SSH to App-B
ewb-rt-a-private	10.10.0.0/16	local	VPC A local route
	10.20.0.0/16	pcx-016ad3738d94d964e	Forward path: App-A to VPC B
ewb-rt-b-private	10.20.0.0/16	local	VPC B local route
	10.10.0.0/16	pcx-016ad3738d94d964e	Return path: App-B to VPC A

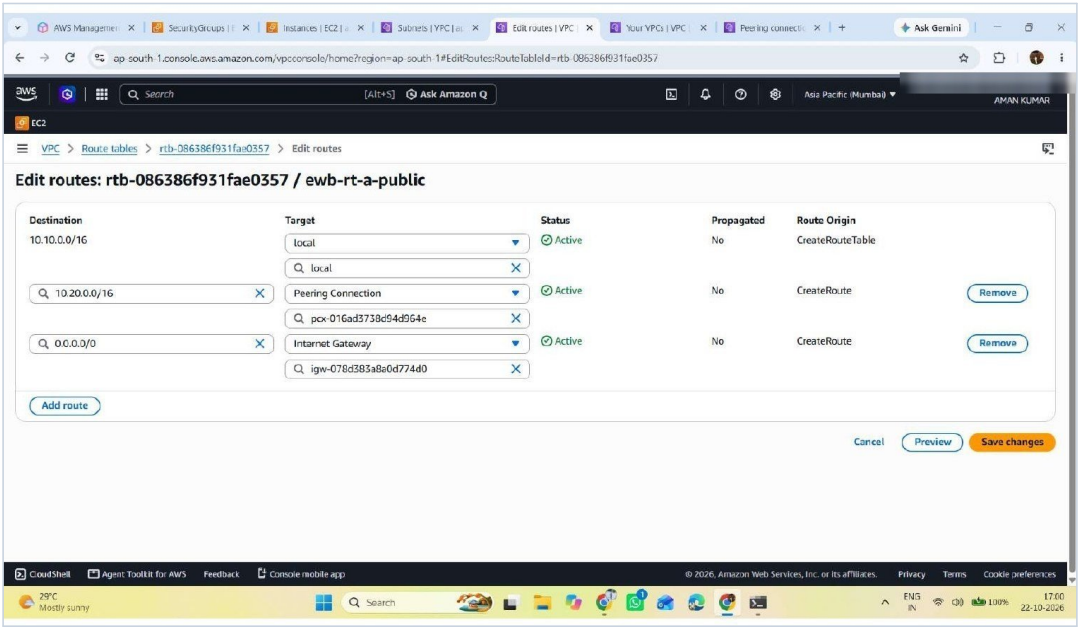


Figure 1. Host: AWS console | Test: Routes of ewb-rt-a-public (local, 10.20.0.0/16 via pcx, 0.0.0.0/0 via igw) | Result: **all three routes Active**

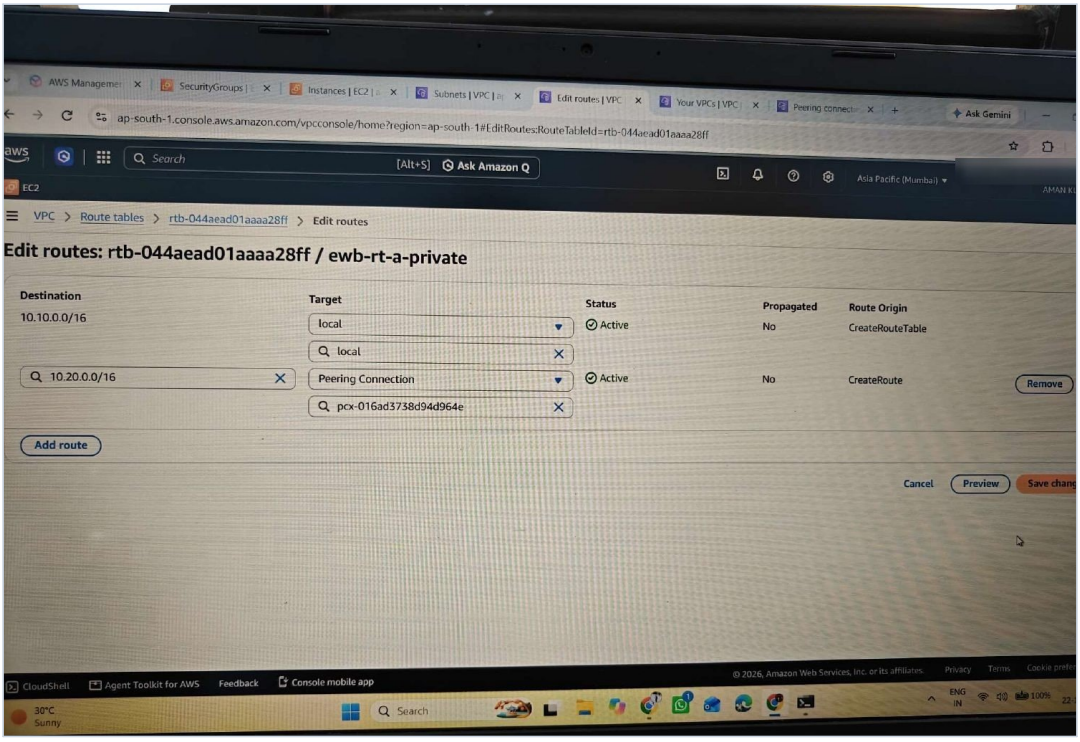


Figure 2. Host: AWS console | Test: Routes of ewb-rt-a-private (local and 10.20.0.0/16 via pcx), no internet route | Result: **routes Active**

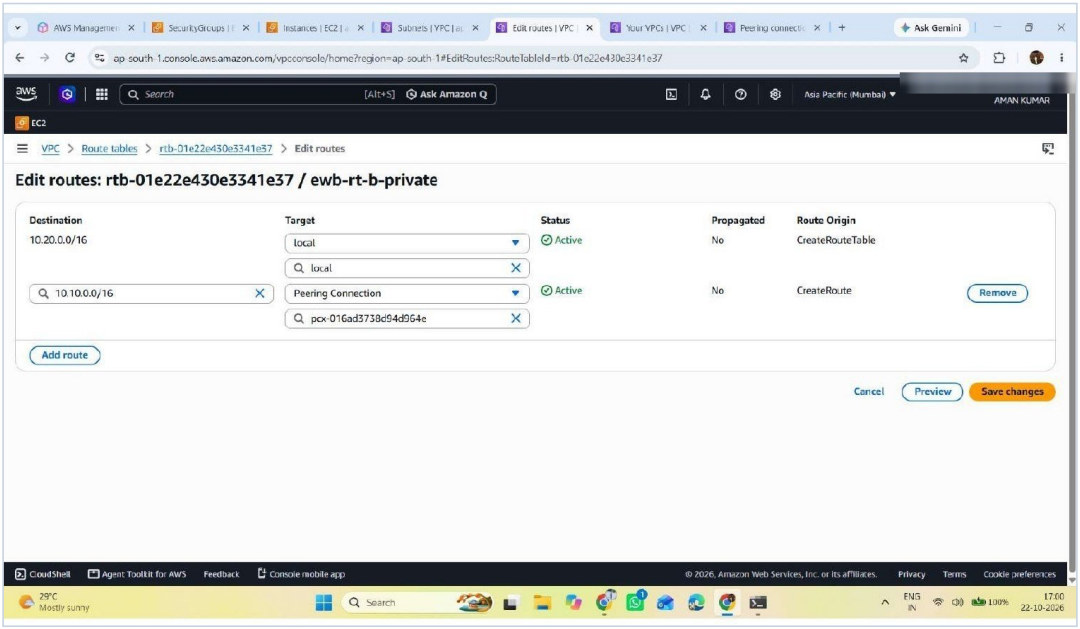


Figure 3. Host: AWS console | Test: Routes of ewb-rt-b-private (local and 10.10.0.0/16 via pcx, the return path) | Result: **routes Active**

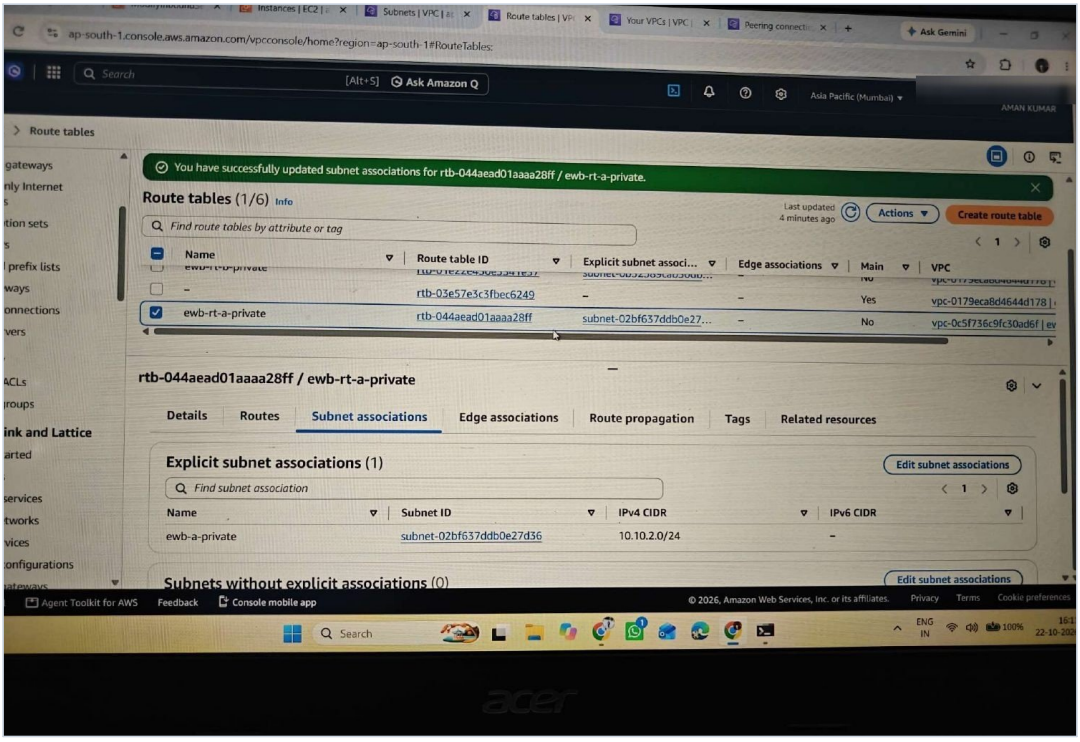


Figure 4. Host: AWS console | Test: Subnet associations of ewb-rt-a-private: ewb-a-private (10.10.2.0/24) explicitly associated | Result: **association saved**

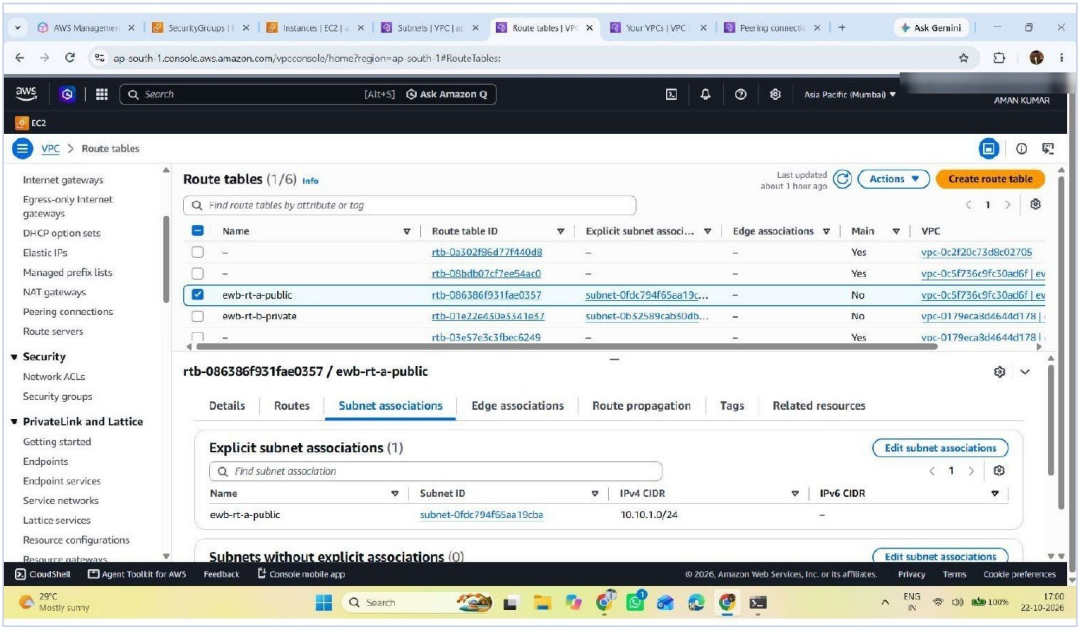


Figure 5. Host: AWS console | Test: Subnet associations of ewb-rt-a-public (ewb-a-public) | Result: **association shown**

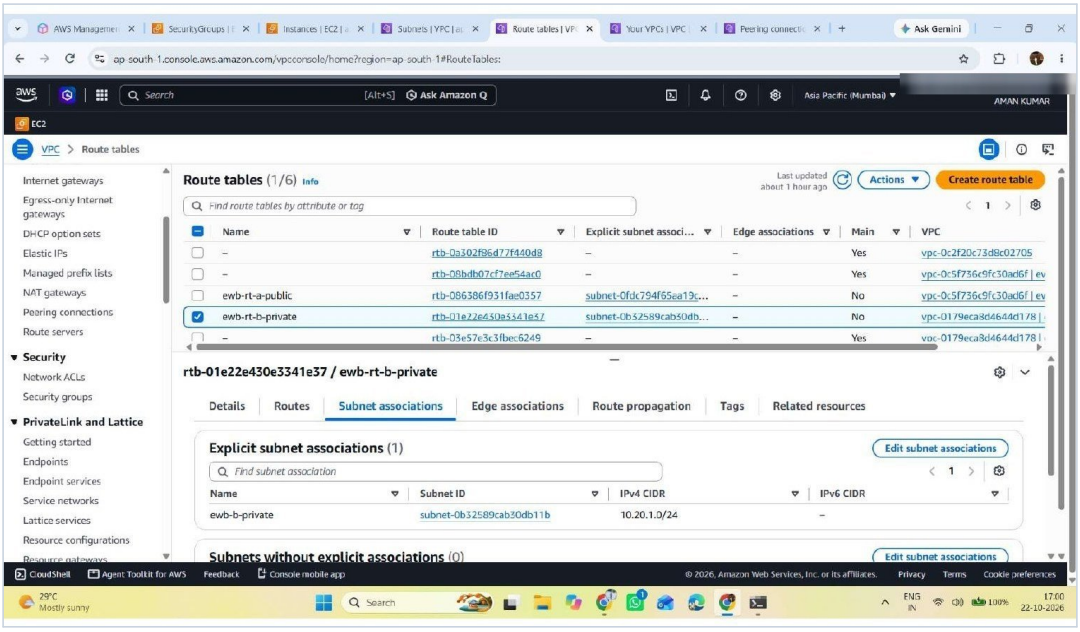


Figure 6. Host: AWS console | Test: Subnet associations of ewb-rt-b-private (ewb-b-private) | Result: **association shown**

Active peering and security group inbound rules

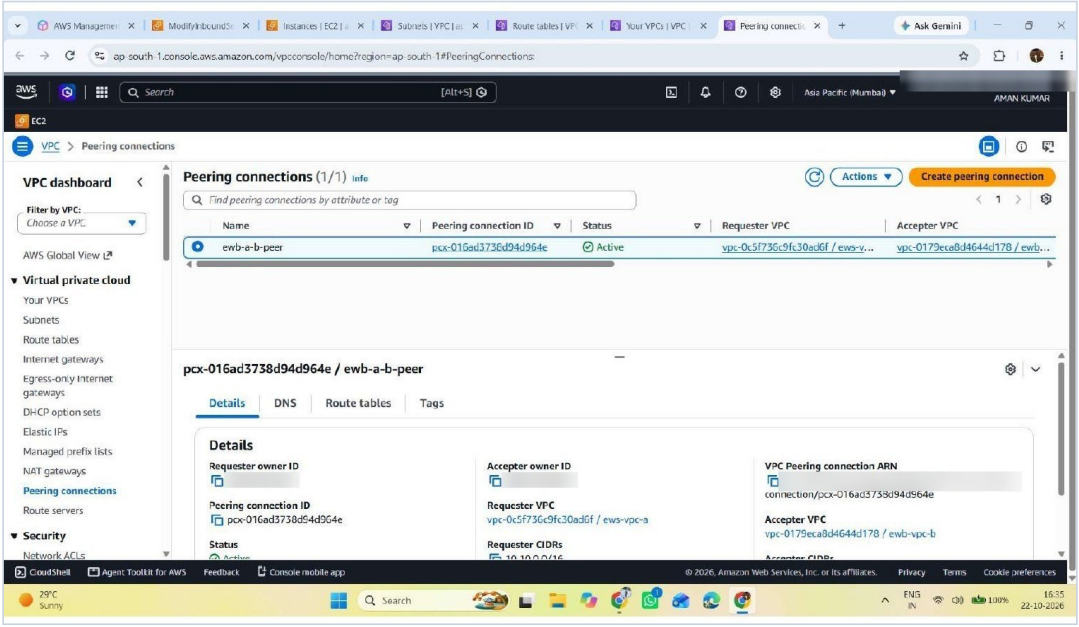


Figure 7. Host: AWS console | Test: Peering connection ewb-a-b-peer, pcx-016ad3738d94d964e | Result: **Status Active, requester VPC A, accepter VPC B**

The Name tag of VPC A appears in this screenshot as `ews-vpc-a`. That was a typing mistake in the tag and was corrected to `ewb-vpc-a`; the VPC ID is unchanged.

Security group	Type / port	Source	Reason
ewb-sg-bastion (VPC A)	SSH / TCP 22	my public IPv4 /32	Only my laptop can reach the bastion
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32	SSH only from the bastion
	Custom ICMP, Echo request	10.20.1.10/32	Ping only from App-B
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32	SSH only from the bastion (ProxyJump source)
	Custom ICMP, Echo request	10.10.2.10/32	Ping only from App-A

Outbound rules stay at the default allow-all. Security groups are stateful, so replies to allowed traffic are allowed automatically. SSH and ICMP are never opened to 0.0.0.0/0.

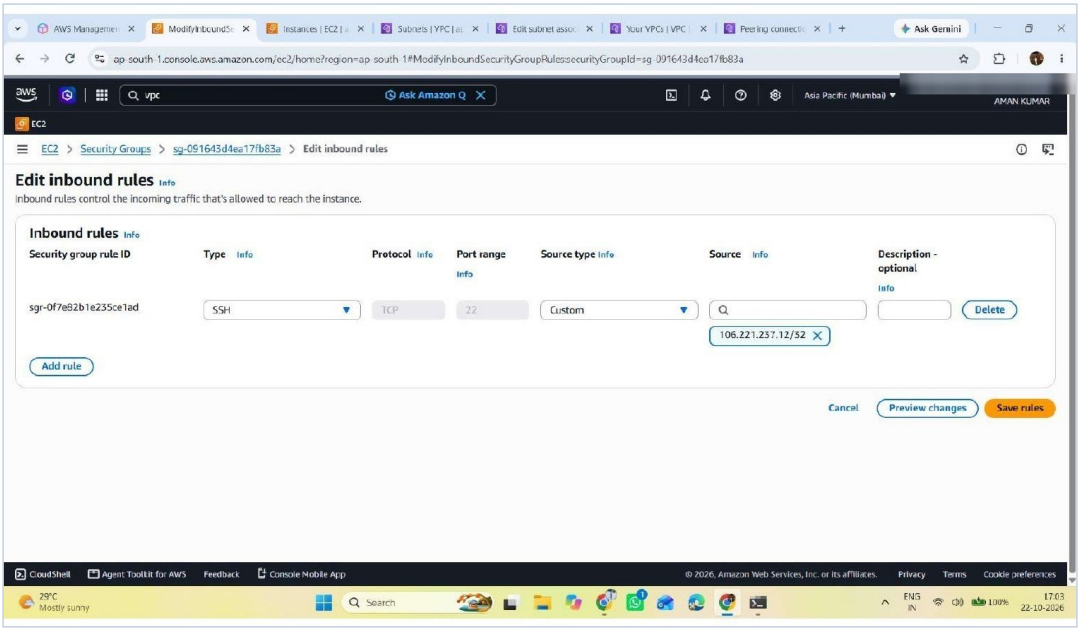


Figure 8. Host: AWS console | Test: Inbound rules of ewb-sg-bastion (SSH 22 from my public IP /32 only) | Result: **single scoped rule**

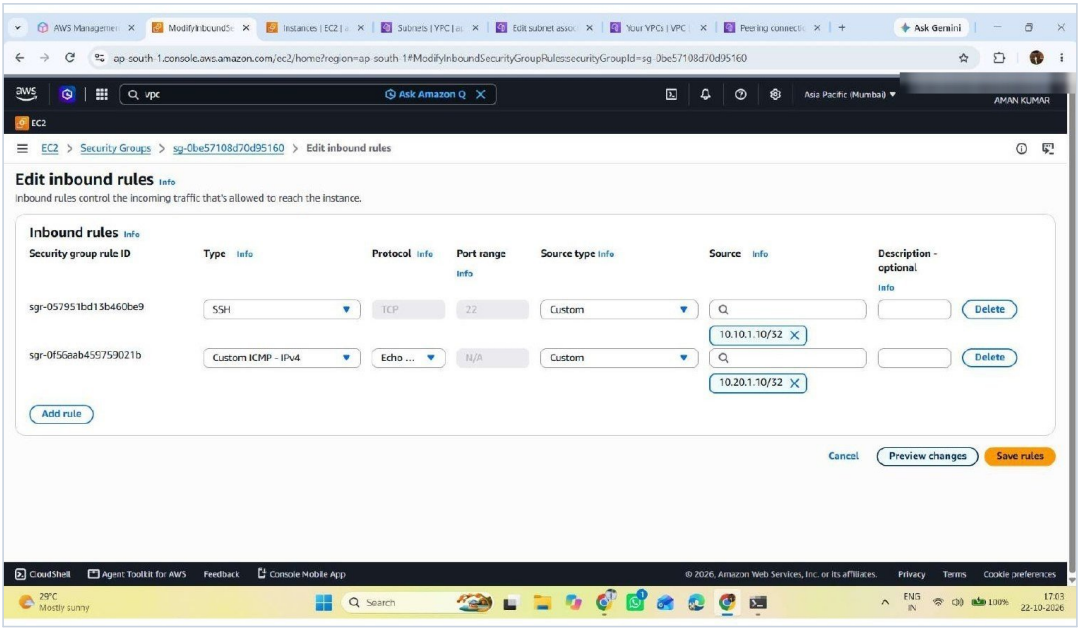


Figure 9. Host: AWS console | Test: Inbound rules of ewb-sg-app-a (SSH from 10.10.1.10/32, ICMP Echo request from 10.20.1.10/32) | Result: **two scoped rules**

Tests with real terminal output

Every test uses private destination IPs. Both private hosts have no public IPv4 address.

5.1 Initial failed test and the fix

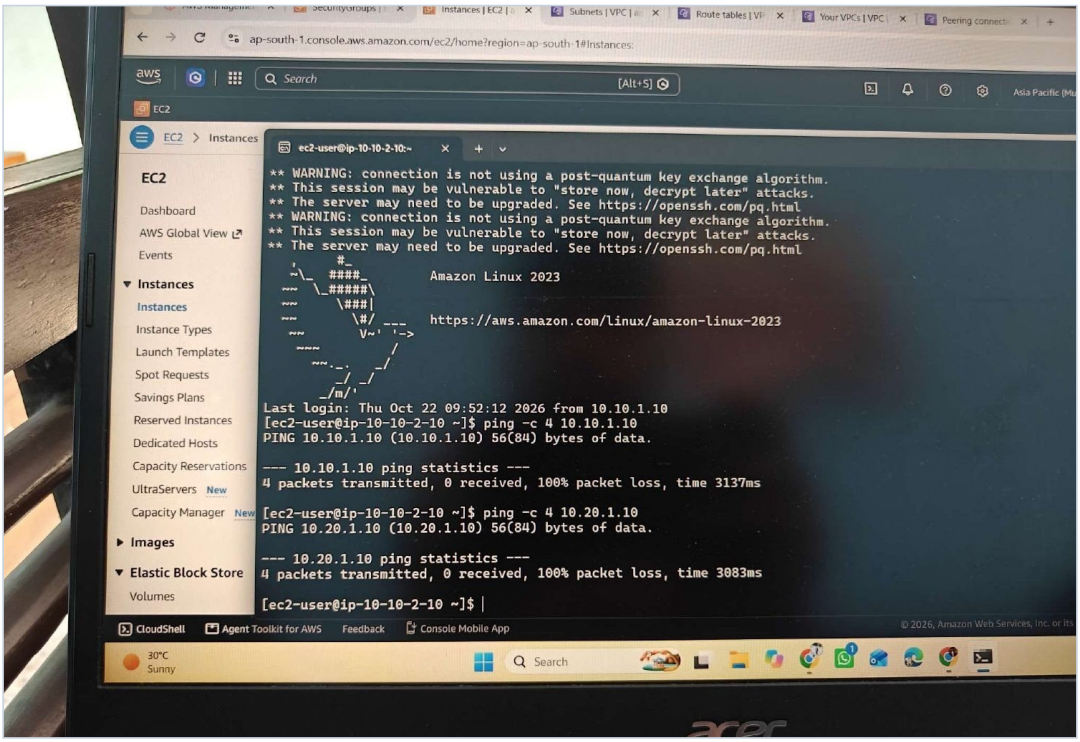


Figure 10. Host: ewb-app-a (10.10.2.10) | Test: ping -c 4 10.20.1.10 | Result: **4 sent, 0 received, 100% packet loss**

Cause found: the peering connection was Active and the routes existed, but subnet `ewb-a-private` was not explicitly associated with `ewb-rt-a-private`. It was still using the main route table, which only has the local route, so App-A had no path to 10.20.0.0/16. Associating the subnet with ewb-rt-a-private fixed it (see the association screenshot in section 3).

5.2 Isolation: no public IP

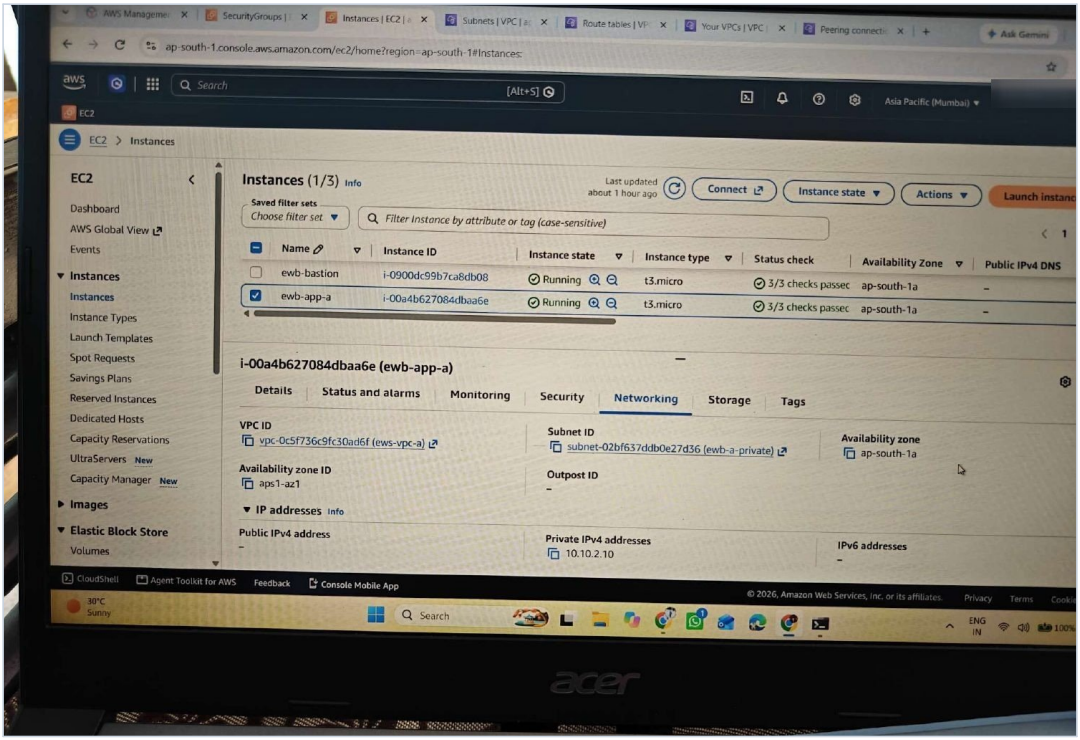


Figure 11. Host: AWS console, ewb-app-a | Test: Instance networking details | Result: **Public IPv4 address empty, private IP 10.10.2.10**

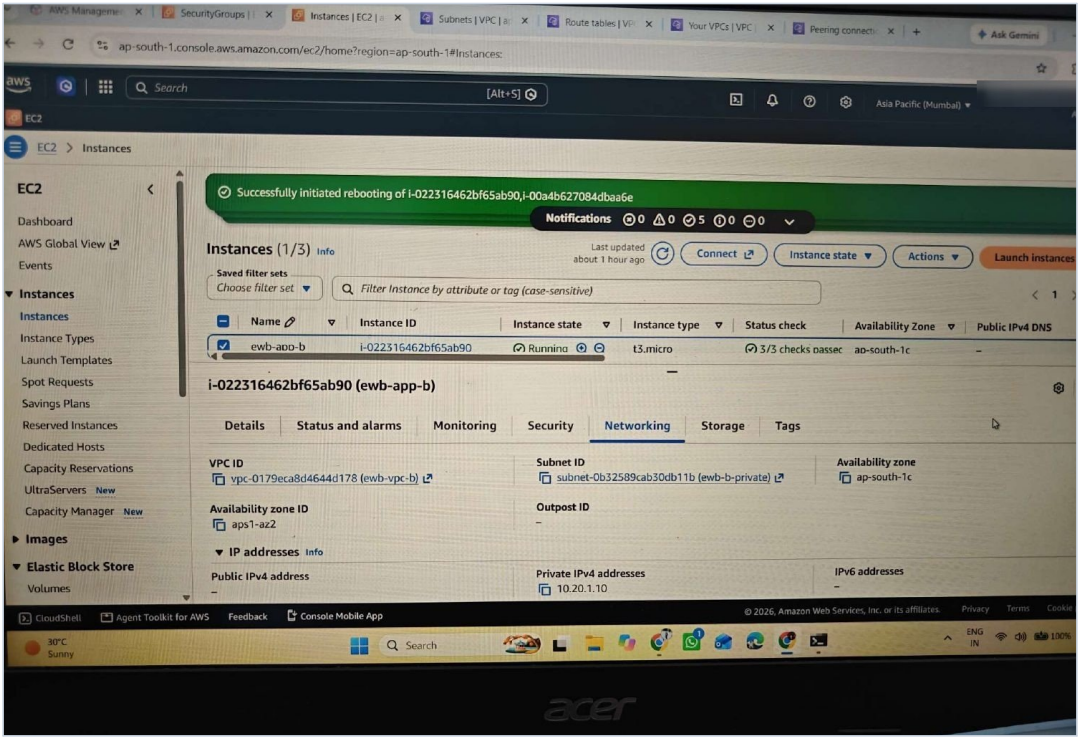


Figure 12. Host: AWS console, ewb-app-b | Test: Instance networking details | Result: **Public IPv4 address empty, private IP 10.20.1.10**

5.3 Supporting evidence

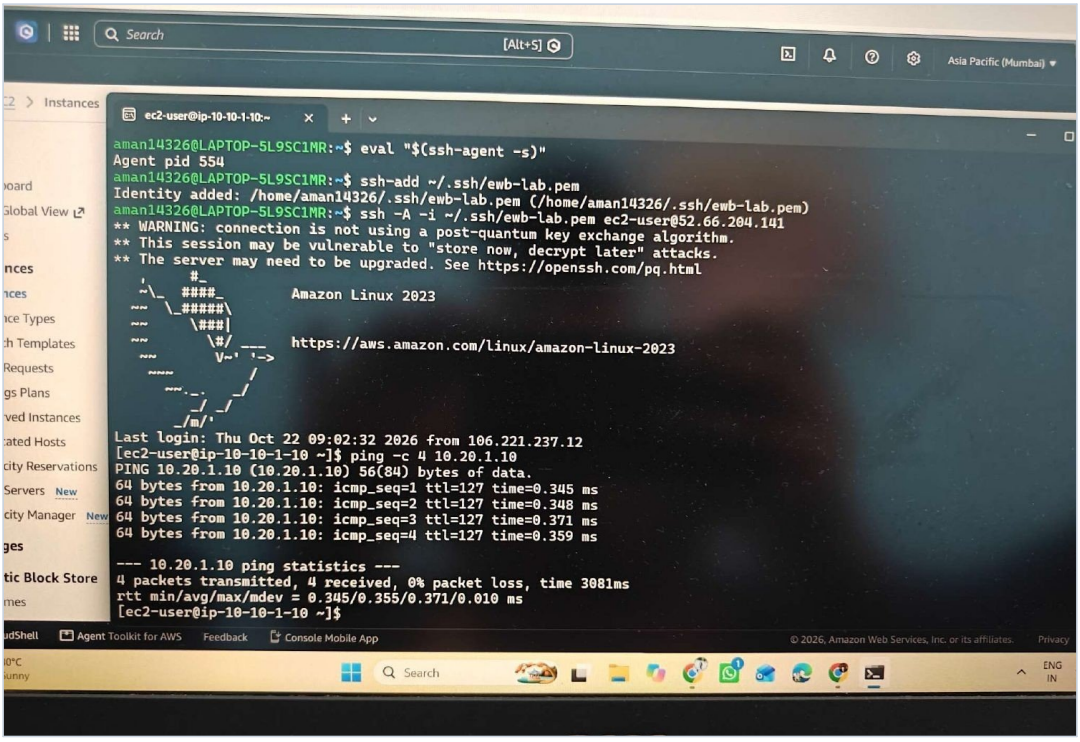


Figure 13. Host: ewb-bastion (10.10.1.10) | Test: ping -c 4 10.20.1.10 | Result: **4 received, 0% packet loss**

This shows the public-subnet route table also reaches VPC B through the peering connection, which is why ProxyJump to App-B works.

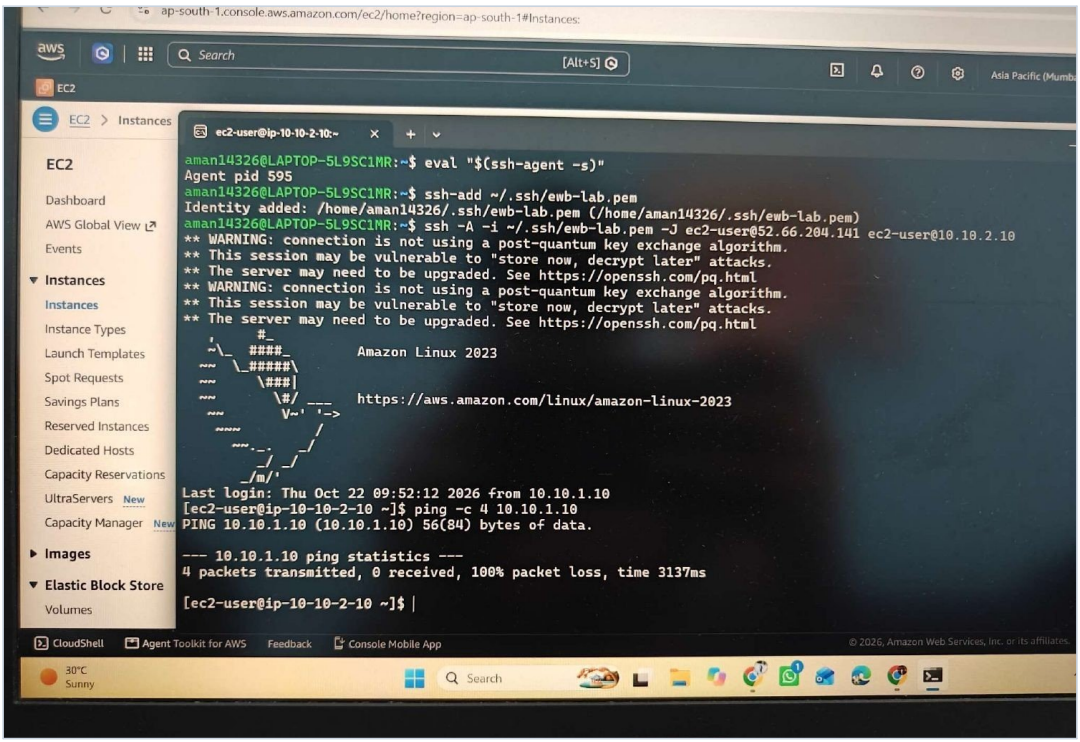


Figure 14. Host: ewb-app-a (10.10.2.10) | Test: ping -c 4 10.10.1.10 (bastion) | Result: **100% loss, expected**

This failure is expected: ewb-sg-bastion allows only SSH, not ICMP. It confirms the security groups are scoped to the lab traffic.

06 / CONTROLLED FAILURE

Remove the rule, see it fail, restore it, see it recover

Explanation of the controlled ICMP failure test (the ICMP Echo request rule on ewb-sg-app-b is the only rule involved; the SSH rule is not touched).

Explanation. A security group is an allow-list. With the Echo request rule removed, no rule would match a ping coming from App-A, so ewb-sg-app-b drops it silently, even though the peering connection is Active and both route tables are correct. When the exact rule is restored, the ping is allowed again and the reply returned over the return route. This shows that peering is only the link: routes decide where traffic can go, and security groups decide whether it is allowed.

Forward path, return path, cleanup

Forward and return path

Forward path (App-A to App-B). App-A (10.10.2.10) sends to 10.20.1.10. The subnet's route table ewb-rt-a-private matches 10.20.0.0/16 and sends the packet to the peering connection pcx-016ad3738d94d964e. ewb-sg-app-b then allows the Echo request from 10.10.2.10/32.

Return path (App-B to App-A). App-B replies to 10.10.2.10. The route table ewb-rt-b-private matches 10.10.0.0/16 and sends the reply back through the peering connection. Without this return route the reply is lost and the ping fails even though the forward path works.

SSH via the bastion. With ProxyJump the bastion opens a new connection to App-B, so App-B sees the bastion's private IP 10.10.1.10 as the source, not my laptop. That is why ewb-sg-app-b allows SSH from 10.10.1.10/32, and why ewb-rt-a-public needs the 10.20.0.0/16 route to the peering connection.

Cleanup evidence

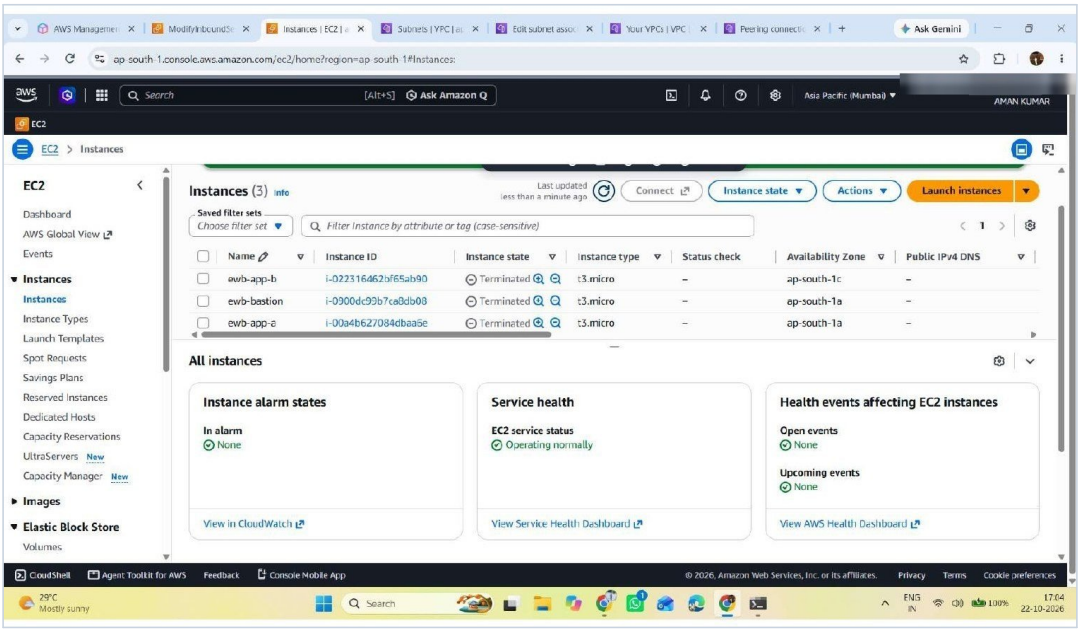


Figure 15. Host: AWS console (EC2) | Test: Instances list after termination | Result: **ewb-app-b, ewb-bastion, ewb-app-a all Terminated**

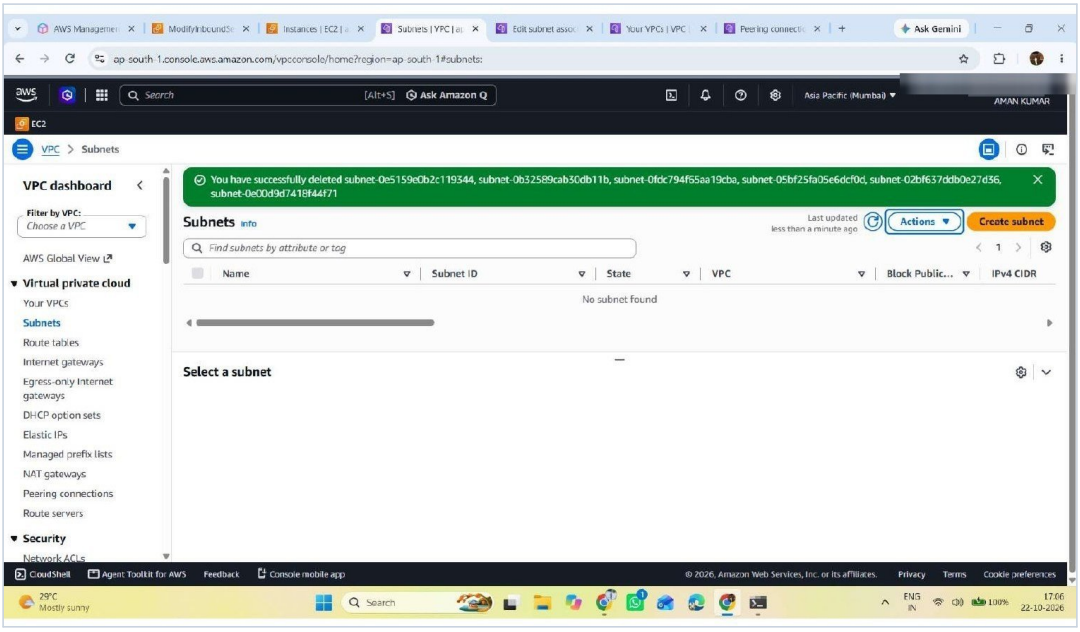


Figure 16. Host: AWS console (VPC) | Test: Delete the lab subnets | Result: **success banner, Subnets list empty (No subnet found)**

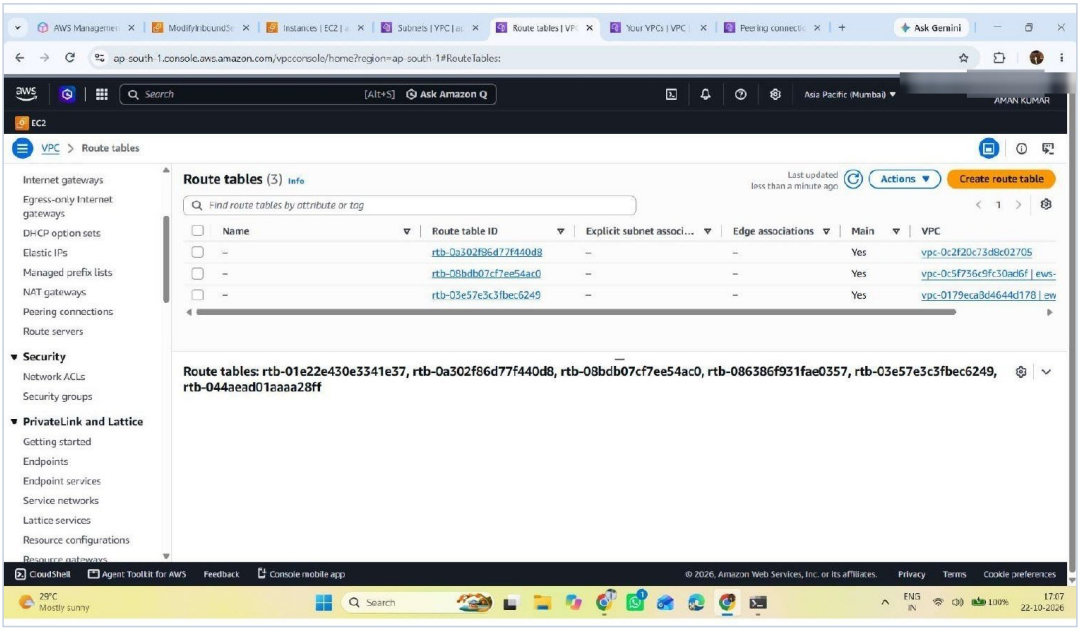


Figure 17. Host: AWS console (VPC) | Test: Route tables after deleting the custom tables | Result: **only the 3 main route tables remain**

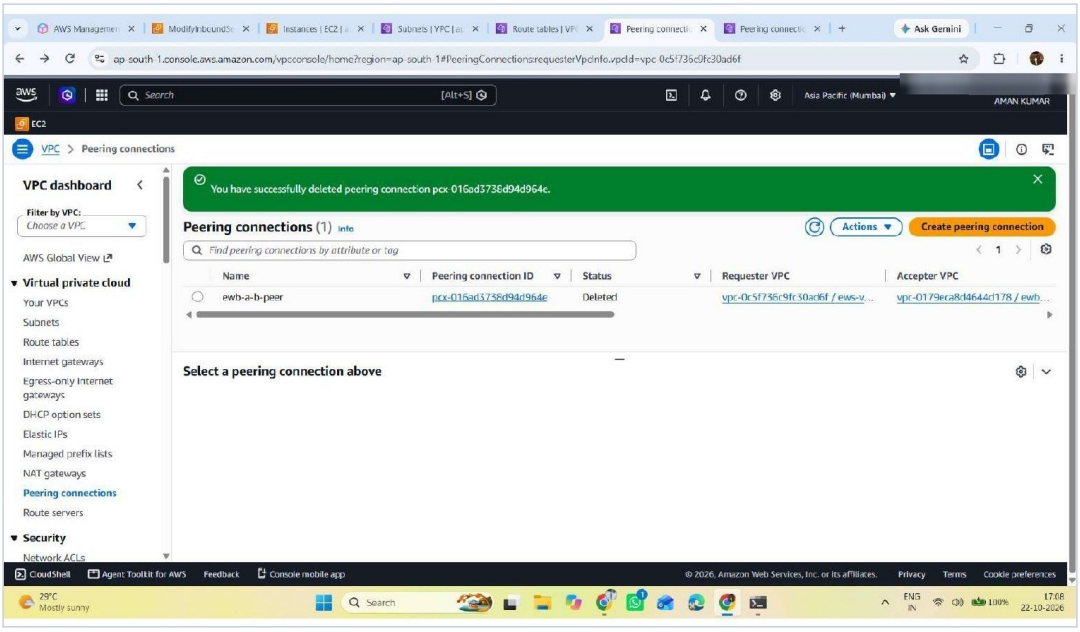


Figure 18. Host: AWS console (VPC) | Test: Delete peering connection pcx-016ad3738d94d964e | Result: **success banner, status Deleted**

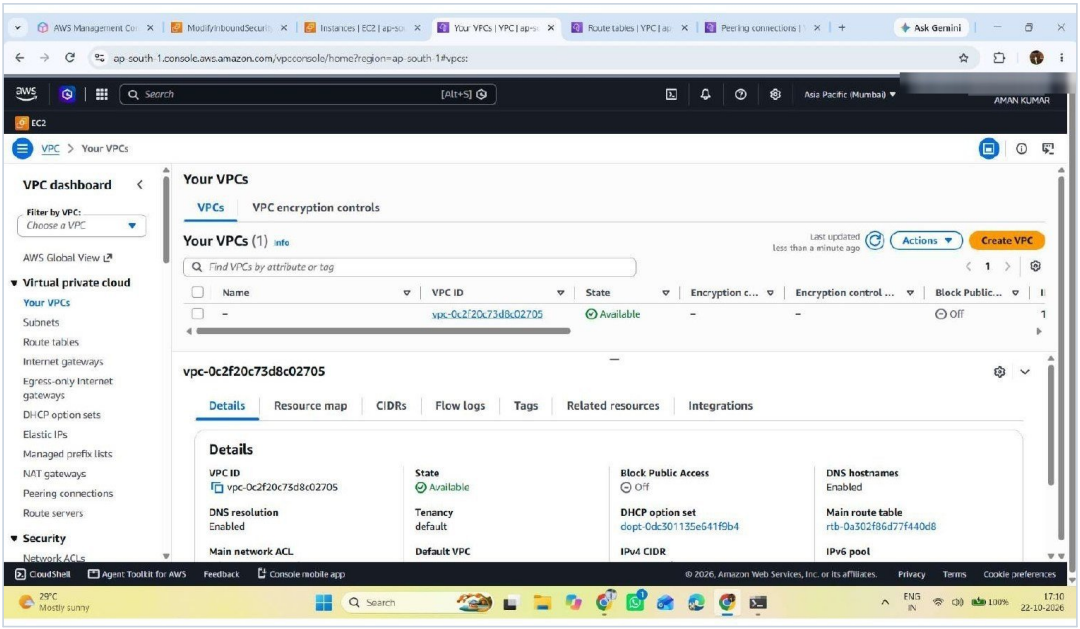


Figure 19. Host: AWS console (VPC) | Test: Your VPCs after deleting ewb-vpc-a and ewb-vpc-b | Result: **only the default VPC remains**

What I learned

1. VPC peering is only the link between two VPCs. Traffic needs a route in both directions, and a security group rule that allows it. 2. A subnet that is not explicitly associated with a custom route table uses the main route table. My App-A ping failed for this reason even though the peering was Active and the route existed in the custom table, so I check associations first now. 3. A bastion with ProxyJump keeps the private servers off the internet. The target server sees the bastion's private IP, so security groups should allow only that /32. 4. Peering is not transitive and does not give VPC B internet access through VPC A. 5. Narrow /32 security group rules also make failures visible: removing one rule stops the ping, and restoring it brings it back.